

Evaluación Parcial 3 - Observabilidad de Agente IA

HealthTech EP3 Observabilidad

Asignatura: Ingeniería de Soluciones con IA | Entrega: EP3

1. Resumen del proyecto

El proyecto implementa observabilidad sobre un agente HealthTech con RAG local y conexión opcional a GitHub Models. La solución registra logs estructurados, trazas, métricas de rendimiento y calidad, además de un dashboard para visualizar el comportamiento del agente durante la ejecución.

La versión final fue ordenada para EP3: se eliminaron archivos de EP2, cachés, evidencias antiguas y documentos duplicados. La entrega queda centrada en métricas, trazabilidad, dashboard, seguridad y recomendaciones de mejora.

2. Arquitectura de observabilidad

El flujo principal comienza cuando el usuario realiza una consulta. El agente aplica una validación de seguridad, crea un trace_id, ejecuta el planner y las herramientas con span_id propios, genera la respuesta y registra el resultado final. Cada paso queda asociado a un evento JSONL para reconstruir la solicitud completa.

Componente	Función	Archivo principal
Agente	Orquesta planner, RAG, memoria, seguridad y respuesta final	app/agent.py
Observabilidad	Genera trace_id, span_id, duración, estado y recursos	app/observability.py
Métricas	Calcula latencia, errores, relevancia, consistencia, anomalías y recursos	app/metrics.py
Dashboard	Visualiza logs, métricas, uso de herramientas, CPU/RAM y anomalías	dashboard.py
Seguridad	Sanitiza datos sensibles y bloquea prompt injection	app/security.py

3. Métricas implementadas

La evaluación mide el comportamiento del agente en escenarios variados mediante métricas de calidad, rendimiento y seguridad.

Indicador	Métrica	Cómo se calcula	Evidencia generada
IE1	Precisión/Relevancia	Comparación de palabras clave esperadas en respuestas de prueba	outputs/metrics/evaluation_results.json
IE1	Frecuencia de errores	Eventos con status failed o event error sobre solicitudes totales	outputs/metrics/metrics_summary.json

IE1	Consistencia	Similitud entre respuestas repetidas a la misma consulta	outputs/metrics/metrics_summary.json
IE2	Latencia	Duración total y por span en milisegundos	outputs/logs/agent_events.jsonl
IE2	Uso de recursos	CPU y memoria registrados al finalizar spans	outputs/metrics/metrics_summary.json
IE4	Anomalías	Trazas con latencia superior al umbral dinámico	Dashboard Streamlit

4. Logs y trazabilidad

Los logs se guardan en formato JSONL para facilitar filtrado y análisis. Cada evento contiene timestamp, level, trace_id, span_id, span, event, user_id, versión del agente, duración y metadatos adicionales. Esto permite reconstruir una petición completa filtrando por trace_id.

Ejemplo de flujo registrado: full_request start -> planner start/end -> tool_memory_context start/end -> tool_rag_search start/end -> full_request end.

5. Dashboard y análisis

El dashboard permite revisar solicitudes, tasa de error, latencia promedio, latencia p95, relevancia, consistencia, logs estructurados, uso de herramientas, CPU, memoria, errores y anomalías detectadas. Se ejecuta con: `python -m streamlit run dashboard.py`

6. Seguridad y uso responsable

La solución incorpora sanitización básica de datos sensibles, bloqueo de instrucciones inseguras y audit trail de decisiones. Para consultas clínicas sensibles, el agente marca necesidad de revisión humana. El sistema no reemplaza criterio médico profesional y se considera apoyo informativo.

7. Hallazgos y mejoras propuestas

- Revisar trazas lentas mediante trace_id para ubicar el span con mayor duración.
- Cachear consultas frecuentes de RAG si la latencia p95 aumenta.
- Ampliar el set de preguntas de evaluación para medir calidad con más cobertura.
- Definir alertas para latencia, tasa de error y consumo de recursos.
- Mantener revisión humana en respuestas clínicas sensibles y registrar decisiones en audit trail.

8. Evidencias y ejecución

Para generar la evidencia final se debe configurar el archivo .env con el token de GitHub Models y ejecutar:

```
pip install -r requirements.txt
```

```
python -m scripts.evaluate_observability
```

```
python -m streamlit run dashboard.py
```

```
python -m pytest -q
```

9. Conclusión

La implementación permite observar el comportamiento interno del agente y deja evidencia técnica para analizar calidad, rendimiento, errores, seguridad y oportunidades de mejora. Con logs estructurados, métricas y dashboard, la solución responde directamente a los requerimientos de EP3 y facilita una revisión clara por parte del evaluador.

Archivos principales para validar: README.md, dashboard.py, app/observability.py, app/metrics.py, scripts/evaluate_observability.py, outputs/logs y outputs/metrics.